Pack 05 · Optimization levers: prefix caching and quantization

Two levers that change the bill: reusing prompt work, and storing or computing in fewer bits. Lessons 03, 04, 06, 11, 14, 22, 40, 46–54, 59. Answer each question out loud in about a minute, then check the card.

Q1 · One dynamic line kills the cache PRIORITY

Lessons: 03, 11, 46, 47

> **INTERVIEWER**
>
> A support assistant sends about 3,300 prompt tokens, 2,900 of them identical across users, but the prefix hit rate is about 1%. The prompt builder puts the request ID and current time on line 2. Why can't the identical tokens after it be reused, what's the fix, and what does it save at about 32 μs per prompt token?

60-second answer

- Each token's K and V depend on **every token before it** (causal attention), so one differing token makes everything after it different, even identical text.
- vLLM enforces this with **chained block hashes**: each 16-token block's hash includes the previous block's, so a miss on line 2 misses every later block.
- Fix: put the request ID and time **at the end** (or out of the prompt). Stable content first, variable content last.
- Prefill: 3,300 × 32 μs ≈ 105 ms before, about 400 × 32 μs ≈ 13 ms after: **≈ 92 ms saved** per request, ≈ 8× less prefill.

Key numbers: a cache hit needs an identical prefix from token 1.

Likely follow-up: "What's it worth on an API bill?" 3,000 cached system tokens plus 200 new input tokens and 300 output tokens at $0.55 / $0.14 cached / $2.19 per million: ≈ $2.42 per 1,000 requests uncached versus ≈ $1.19 cached. Output tokens are never discounted.

> **TRAP**
>
> I explained why the ID's own block misses but not why the identical tokens after it miss. The time saved is the difference (105 − 13 ms); the ratio is the speedup.

Q2 · 80% hits, but not 80% faster PRIORITY

Lessons: 47, 04

> **INTERVIEWER**
>
> With an 80% prefix hit rate, the PM expected TTFT to drop 80% and responses to get 5× faster. Instead mean TTFT went 108 → 68 ms, end-to-end time for long answers barely moved, and a batch job with 49% hits got 1.82× input throughput. Explain all three.

60-second answer

- Caching removes only the **cached part of prefill**. TTFT also has fixed costs (scheduling, the uncached tail, the first decode step), so it falls less than the hit rate.
- Long answers are dominated by **decode** (e.g. 1,000 output tokens), which caching doesn't touch.
- It's Amdahl: the prefill-only ceiling is **1 ÷ (1 − hit rate)** = 1 ÷ 0.51 ≈ **1.96×**, and 1.82× is close to it.

Key numbers: max prefill speedup = 1 ÷ (1 − h).

Likely follow-up: "When does prefix caching matter most?" Long shared prompts with short outputs: RAG, agents with big tool schemas, multi-turn chat.

> **TRAP**
>
> I explained the directions but not why TTFT fell less than the hit rate (fixed costs), and didn't give the 1 ÷ (1 − h) ceiling.

Q3 · Ideal 75%, measured 30% PRIORITY

Lessons: 46, 48

> **INTERVIEWER**
>
> Replaying an hour of real prompts with the server's chained 16-token hashing gives an ideal hit rate of 75%. vLLM's hits ÷ queries over the same hour gives 30%. What causes the gap? And if the replay were low too, how do you find the line to fix?

60-second answer

- The replay assumes **unlimited memory, one cache and the same fingerprinting**. A high ideal with low measured means the **server** loses the reuse:
- the cache is too small, so blocks are **evicted** before they come back;
- a different **chat template or tokenizer**, or a per-tenant cache salt;
- **several replicas behind a random load balancer**, each seeing only a slice of the repeats.
- If the replay is low too, the prompts break the prefix. Plot a **first-divergence histogram**: where each request first differs from the previous one from the same app. A spike at a low position points at the dynamic line.

Key numbers: none; it's a diagnosis.

Likely follow-up: "Which metric shows eviction?" KV cache usage pinned high with the hit rate falling under load; compare hit rates at low and high load.

> **TRAP**
>
> I couldn't start. The way in: list what the replay assumes, then ask which assumption the real server breaks.

Q4 · Multi-turn chat, then trimming PRIORITY

Lessons: 14, 40, 48

> **INTERVIEWER**
>
> A chat has a 1,000-token system prompt; each turn adds a 100-token question and a 300-token answer. Over 6 turns, how many prefill tokens without reuse and with a perfect prefix cache? At turn 7 the app starts trimming the oldest message every turn: what happens?

60-second answer

- No reuse: turn n re-sends $1{,}000 + 100n + 300(n-1)$: $1{,}100 + 1{,}500 + \ldots + 3{,}100 =$ **12,600 tokens**, growing with the square of the turns.
- With reuse the whole previous conversation is cached, **including the model's own answers** (their K and V were computed while generating). Each turn computes only the new question plus a partial block: $\approx$ **1,640 tokens**, about 7× less and linear.
- Trimming the oldest message changes tokens right after the system prompt, so **everything after it misses** every turn ($\approx$ 2,100 tokens recomputed instead of $\approx$ 100).
- Better: stay **append-only**; when you must cut, cut a big chunk rarely, or summarize once.

Key numbers: prefix caching turns a conversation's quadratic prefill into linear.

Likely follow-up: "What happens to a finished request's blocks in vLLM?" They go to the free queue but keep their cached KV, so the next turn can reuse them until they're overwritten (deepest blocks first, since they're the least shared).

> **TRAP**
>
> "Keep the oldest" was too vague. Say it: any change near the start costs a full recompute, so trim rarely and in bulk.

Q5 · RAG order, replica routing and tenant isolation PRIORITY

Lessons: 47, 48

> **INTERVIEWER**
>
> A RAG prompt is 800 tokens of instructions, 5 passages × 300 tokens, then a 50-token question. Two users get the same passages in a different order. How much is reused, and how do you get near 100%? What happens on 4 replicas behind round-robin, and how do you stop customers A and B from sharing cached K/V?

60-second answer

- Different order: only the 800 instruction tokens match, ≈ **34%** of 2,350. Sort passages by a **stable key** (e.g. document ID) and both share ≈ 2,300 tokens, ≈ **98%**.
- Round-robin on 4 replicas: the cached copy is on the right replica only ≈ **1 in 4** times. **Route by prefix hash** (or session ID for chats), not by request ID, which is unique per request.
- Only **full 16-token blocks** are cached: of a 1,500-token prompt, 16 × ⌊1,500 ÷ 16⌋ = **1,488** tokens are reusable.
- Isolation: a per-tenant **cache_salt** mixed into the hash. Same text, different tenants, no match, and caching stays on.

Key numbers: reusable = 16 × ⌊tokens ÷ 16⌋.

Likely follow-up: "Downside of prefix routing?" Hot prefixes overload one replica; you need load-aware fallbacks.

> **TRAP**
>
> I said route "by request ID or some hash". A request ID scatters requests just like round-robin; the key must be the shared prefix.

Q6 · RadixAttention, eviction and churn PRIORITY

Lessons: 47, 49

> **INTERVIEWER**
>
> Two prompts share their first 47 tokens: how many does vLLM reuse, and SGLang's RadixAttention? Under memory pressure, what does SGLang evict? And a cache holds 4 prefixes while traffic cycles through 5 (A to E, twice): how many hits under LRU?

60-second answer

- vLLM reuses only full blocks: **32** tokens. RadixAttention matches token by token: all **47**.
- SGLang never evicts nodes **in use** (reference count > 0). Among the rest it evicts **LRU leaves** first, so shared trunks like the system prompt survive longest.
- Cycling 5 prefixes through a 4-entry LRU: each one is evicted just before it returns, so **0 of 10 hit**. One more slot gives 5 of 10.
- **Cache-aware scheduling** (serve the longest cached prefix first) runs a group while its prefix is still cached. The risk is **starving** requests with no shared prefix.

Key numbers: a working set just above cache size gives near-zero hits under LRU.

Likely follow-up: "How would you fix churn without more memory?" Group or reorder requests by prefix, and pin the hottest prefixes.

> **TRAP**
>
> I missed the pinning rule (in-use nodes are untouchable) and called starvation "no space for shorter ones".

Q7 · Outliers, AWQ, GPTQ and SmoothQuant PRIORITY

Lessons: 51, 52

> **INTERVIEWER**
>
> Why do a few outlier values make low-bit quantization hard, and how do AWQ, GPTQ and SmoothQuant each work around the problem?

60-second answer

- **One large value sets the group's scale**, so the grid stretches and the small values round to zero. Small groups (128 weights, or NVFP4's 16) limit the damage.
- **AWQ** (weight-only, 4-bit): about 1% of weight channels matter most, found by their **large activations**. Scale those weights up before rounding so they get a finer grid, and divide the activations by the same factor.
- **GPTQ** (weight-only): quantize column by column and push each rounding error onto the **weights not yet quantized**, so errors cancel. 175B in ≈ 4 GPU-hours.
- **SmoothQuant** (W8A8): **activations** have the outliers. Divide each activation channel by s and multiply the weights by s, so both fit INT8 and the **math runs in 8 bits** (≈ 1.5× faster, half the memory).

Key numbers: group of 128 with a 16-bit scale costs 4 + 16 ÷ 128 = 4.125 bits per weight.

Likely follow-up: "How lossy is each format?" FP8 is essentially lossless; INT8 W8A8 loses ≈ 1–3%; good 4-bit keeps ≈ 99% on most benchmarks.

> **TRAP**
>
> I said AWQ quantizes activations too (it's weight-only) and missed what SmoothQuant unlocks (8-bit math).

Q8 · Why 4-bit stops paying off at big batches PRIORITY

Lessons: 50, 51, 53, 59

> **INTERVIEWER**
>
> A 4-bit AWQ model with a Marlin kernel is about 3.5× faster than BF16 at batch 1, about 1.5× at batch 128, and break-even on long prefills. Why? Why would a kernel that expands the whole matrix to 16 bits first be slower than plain BF16? What would you use for a throughput-heavy workload on H100s?

60-second answer

- Batch 1 is limited by **reading weights**: 4× fewer bytes gives nearly 4× (≈ 3.9× with scales).
- Bigger batches reuse each weight for more tokens, so the limit moves to **math**, which still runs in 16 bits, plus unpacking every tile.
- Naive expand-first reads 0.5 B, writes 2 B, reads 2 B again: **4.5 bytes per weight** versus 2. Good kernels unpack **in registers** and never write 16-bit weights back.
- For throughput use **FP8 W8A8**: both inputs go straight into 8-bit tensor cores, scales applied once at the end, ≈ **2× the math** (1,979 vs 989 TFLOPS). So: 4-bit weight-only for a latency-first chatbot, FP8 W8A8 for an overnight batch job.

Key numbers: 4-bit weight-only saves bytes; 8-bit weights + activations saves math. Prefill of 2,000 tokens ≈ 32 ms of math against < 5 ms of weight reading.

Likely follow-up: "A teammate tested 4-bit with 2,000-token prompts and `max_tokens=1`, saw no change, and wants to drop it." That only measured prefill (TTFT), which is compute-bound. Benchmark decode instead: short fixed prompt, long answer, compare TPOT. Judge prefill by MFU and decode by MBU; a decode step at 1% MFU is normal.

> **TRAP**
>
> I couldn't start. Ask first: is this workload limited by bytes or by math?

Q9 · "Only 0.8 points, ship it" PRIORITY

Lessons: 06, 22, 54

> **INTERVIEWER**
>
> The 4-bit model scores 71.2% against BF16's 72.0% on 1,000 questions, and the PM says "only 0.8 points, ship it." What checks would you run first, and why isn't the score enough?

60-second answer

- **Averages hide flips**: answers can go right → wrong and wrong → right with the same score. Microsoft found accuracy moving ≤ 2% while up to 13.6% of answers flipped.
- **Distance**: same top token and KL divergence against the original. No labels needed, minutes to run, catches broken conversions.
- **Noise floor**: run the baseline against itself in the same engine under load; batching alone changes answers. Only flips above the floor count.

- **Paired error bars**: each score alone is ±2.8 points, so compare question by question. With 34 wrong → right and 42 right → wrong, the difference is −0.8 ± 1.7: not significant, but read the 42.
- **Your product**: a few hundred real prompts with your template and lengths, judged side by side. Decide with a rule written **before** the run.

Key numbers: $\pm 1.96 \times \sqrt{(p(1-p) \div n)} \approx \pm 2.8$ at $n = 1{,}000$, $p \approx 0.72$.

Likely follow-up: "Temperature 0 makes it deterministic, so why a noise floor?" Batch size changes the arithmetic: 1,000 runs of one prompt at $T = 0$ gave 80 different completions.

> **TRAP**
> I named only flips and the product eval, and missed the paired error bars.

Q10 · "4-bit loses 10 points" vs 98.9% recovered PRIORITY

Lessons: 51, 54

> **INTERVIEWER**
> A blog post says 4-bit Llama 3.1 loses 10 points on coding; Red Hat's 500,000-evaluation study found 4-bit recovers 98.9% on HumanEval. What explains the gap, and does 98.9% mean users get the same answers?

60-second answer

- **Mismatched chat template** or system prompt: the model is hurt by the setup, not by quantization.
- **Over-sensitive scoring**: exact-match or format checks mark a correct but reworded answer wrong. Small evals add noise.
- **Recovery isn't sameness**: it compares averages, and many answers can still flip. Check flips against the noise floor and run your own prompts.

Key numbers: 8-bit > 99% recovered; 4-bit 98.9% on HumanEval (Red Hat, Llama 3.1).

Likely follow-up: "Your eval shows a big drop. First thing you check?" The chat template and the scoring script, before blaming the weights.

> **TRAP**
> I couldn't start. Before blaming the model, ask whether the test itself was set up correctly.

Build it yourself

- `exercises/02_prefill_tokens.py`: counts fleet prefill tokens for round-robin versus prefix-aware routing across 4 replicas. Run `uv run exercises/02_prefill_tokens.py`.